Lab 8 – Backend vs. Frontend

Backend Sheet (pair lab) 100 minutes

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Goal

You work in pairs and build a **Collaborative Playlist** web app: users see the songs, add a song and vote for a song. **Together**, you write the API contract in `CONTRACT.md`. **Alone**, one of you builds the *backend* (`backend.py`, FastAPI) and the other the *frontend* (`index.html`). **Together**, you put the two halves in one folder and run the app. You both get the mark of the whole app.

Your role

You are the **backend person**. You build the server in `backend.py` with FastAPI. It already runs on `http://127.0.0.1:8000`, serves the page (`index.html`) there, and holds the three starting songs. Keep `create_app()`, the route for `/` and the line `app = create_app()`.

Set up (both of you)

1. Pair up with one other student. Decide who builds the backend and who builds the frontend.
2. Extract `lab08_playlist_files.zip`. Move the folder `lab08_playlist` into your own `ai1220` repository.
3. In `lab08_playlist`, run:

   ```
   uv sync --frozen
   uv run python backend.py
   ```

   Open `http://127.0.0.1:8000`: you see the empty playlist page. Stop the server with `Ctrl+C`.
4. Fill in `TEAM.md` with both names, GitHub usernames and roles.

Step 1: Write the API contract (together)

Agree with your partner how the page and the server talk. The app needs three endpoints:

1. get all songs (the server starts with the three songs below);
2. add a song with a title and an artist (refused if one is missing or empty);
3. vote for a song (refused if there is no song with that id).

id	Title	Artist	Votes
1	Blue Monday	New Order	3
2	Clocks	Coldplay	1
3	Dancing Queen	ABBA	0

Fill in every line of `CONTRACT.md`: the method, the path, the request body, the response body and the status code of each endpoint, and the error status and error body of adding and voting. Rules:

- Write paths without host or port, such as `/things`. In the vote path, write the song's id as `{id}`, such as `/things/{id}/likes`.
- Use `GET` to read data, and `POST`, `PUT`, `PATCH` or `DELETE` to change it. Write status codes as numbers, such as `201`.
- Use the same error body for both refusals, with a message for the user, such as `{"error": "There is no song 99"}`.
- Use one naming style for all fields: `camelCase` or `snake_case`.

Be exact with field names: if one of you writes `song_name` and the other `title`, nothing works.

One of you types the contract. Send `CONTRACT.md` to your partner (AirDrop, Teams or email), so you both have the same file. Change it later only if you both agree, and send the new file again.

Step 2: Build the server (backend person)

Write your endpoints inside `create_app()`, following the contract.

1. Rename the fields of the starting songs to the names in your contract.
2. **Get all songs:** return the songs in the order they were added.
3. **Add a song:** check that the title and the artist are not missing or empty. If they are fine, store the song with the next id and 0 votes, and return the contract's response. If not, return the contract's error and store nothing.
4. **Vote:** add one vote to the song with that id. If there is no such song, return the contract's error.

Read the request body yourself: add `request: Request` to your function (`from fastapi import Request`) and use `body = await request.json()`. Do not use a Pydantic model here: it refuses bad input with its own 422 body, not the contract's error. For a status other than 200, use `@app.post("/...", status_code=201)`, or `return JSONResponse(status_code=400, content={...})` (from `fastapi.responses`).

Run `uv run python backend.py` and test each endpoint at http://127.0.0.1:8000/docs with **Try it out**. Compare the status and the body with the contract. Saving `backend.py` restarts the server with the three starting songs.

Step 3: Run the app (together)

1. Send your file to your partner: the backend person sends `backend.py`, the frontend person sends `index.html`. Put your partner's file in your `lab08_playlist` folder, replacing the old one. Now you both have the whole app.
2. Start the server with `uv run python backend.py` and open http://127.0.0.1:8000.
3. Open the Network tab of the browser's developer tools (Chrome: `Cmd+Option+I`).
4. Test everything: the three songs appear; a new song appears; a vote adds one; an empty song shows the error message.
5. If something fails, compare the request and the response with `CONTRACT.md`. The person who owns the wrong file fixes it and sends it again.

At the end, you both have the same `CONTRACT.md`, `backend.py` and `index.html`.

Submission and marking

Each of you pushes the final `lab08_playlist` folder to your own `ai1220` repository before the end of the lab. Commit your own file during Step 2 too, so your work shows in your history. Run these in the top folder of your `ai1220` repository (one level above `lab08_playlist`):

```
git add lab08_playlist
git commit -m "Lab 8 playlist"
git push
```

Part	Weight	What is checked
Contract	30%	Every line is filled in and follows the rules of Step 1.
Backend	35%	On a fresh server, each endpoint answers as the contract says: the list, adding (including a song with other values), refusing an empty song without storing it, voting, and refusing a vote for song 99999.
Page	35%	With your server: one card per song, in the server's order; adding shows the new card and empties the fields; voting shows one more vote; an empty song shows the server's error message. Before testing the page, we add a fourth song through the server, so do not write the songs into the page.